

INORDER TRAVERSAL (NON-RECURSIVE STACK IMPLEMENTATION)

Exact Original Code Structure with Line-by-Line Explanatory Comments

```
// Online C compiler to run C program online
#include <stdio.h> // Standard I/O library for printf/scanf
void push(int a); // Function prototype for push
int pop(); // Function prototype for pop
int stack[100]; // Array-based auxiliary stack to store tree indices
int top = -1; // Top of stack initialized to -1 (empty stack)
void push(int a) // Push function definition
{
    top=top+1; // Increment top index
    stack[top]=a; // Store node index 'a' at top of stack
}
int pop(){ // Pop function definition
    int a=stack[top]; // Retrieve element from top of stack
    top=top-1; // Decrement top index
    return a; // Return popped node index
}
int main() { // Main entry point
    printf("enter the number of elements in the binary tree\n"); // Prompt for total node count
    int n; // Variable for array/tree capacity
    scanf("%d",&n); // Read total elements count
    int a[n]; // Array representation of binary tree (level-order)
    printf("enter the elements\n"); // Prompt for tree node values
    for(int i=0;i<n;i++) // Loop to input all elements
    {
        scanf("%d",&a[i]); // Store tree nodes in array
    }

    int i=0; // Start traversal from root index 0
    while (top!=-1 || (i<n && a[i] !=0)) // Continue if stack has nodes or current subtree is non-empty
    {
        while (i<n && a[i]!=0){ // Reach the leftmost node of current subtree
            push(i); // Push current root index onto stack
            i=2*i+1; // Move to left child index (2*i + 1)
        }
        i=pop(); // Pop leftmost unvisited node
        printf("%d->",a[i]); // Visit node (Inorder: Root after Left subtree)
        i=2*i+2; // Move to right child index (2*i + 2)
    }
    return 0; // Program finished
    // Write C code here
    return 0;
}
```